

Laboratory Report: Computational Complexity of Linear Equation Systems using LU Decomposition

Vladyslav Humeniuk

1 Introduction

The primary objective of this experiment was to empirically verify the theoretical computational complexity of solving systems of linear equations of the form $Ax = b$ using LU decomposition. Theoretical analysis suggests that the decomposition of a non-singular matrix A into lower (L) and upper (U) triangular matrices requires approximately $\frac{2}{3}N^3$ floating-point operations, corresponding to a time complexity of $O(N^3)$ [1]. Conversely, the subsequent step of solving the system via forward and backward substitution requires $O(N^2)$ operations.

This report presents a performance analysis of these two distinct phases using the *GNU Scientific Library* (GSL)[2], measuring execution times for matrix dimensions ranging from $N = 10$ to $N = 1000$.

2 Methodology

The experiment was implemented in C using the GSL routines `gsl_linalg_LU_decomp` and `gsl_linalg_LU_solve` described in the library documentation[2]. The simulation procedure was as follows:

1. **Data Generation:** For each $N \in [10, 1000]$, a matrix A and vector b were populated with random values from a uniform distribution.
2. **Decomposition:** The time required to compute the LU factorization was measured.
3. **Solution:** The time required to solve the system using the pre-computed factorization was measured.
4. **Verification:** The solution was validated by computing the Euclidean norm of the residual vector $r = \|Ax - b\|$.

Time measurements were performed using `getrusage()` to capture CPU time (user + system) with microsecond precision.

3 Results

The collected performance data is summarized in Table 1. The residual errors for all test cases were within the order of machine precision (10^{-10} to 10^{-13}), confirming the numerical stability of the algorithm provided by GSL[2].

Table 1: Execution times and residual errors for selected matrix sizes.

N	Decomposition Time [s]	Solving Time [s]	Error $\ Ax - b\ $
10	0.000206	0.000026	5.68×10^{-13}
100	0.001146	0.000061	3.93×10^{-12}
500	0.030898	0.000285	1.83×10^{-10}
600	0.045422	0.000410	2.04×10^{-09}
800	0.104717	0.000754	2.00×10^{-10}
1000	0.201480	0.001222	1.20×10^{-09}

3.1 Complexity Analysis

The empirical data clearly illustrates the disparity in growth rates between the two phases. Figure 1 visualizes the decomposition time, which exhibits a cubic growth trend consistent with numerical theory[1].

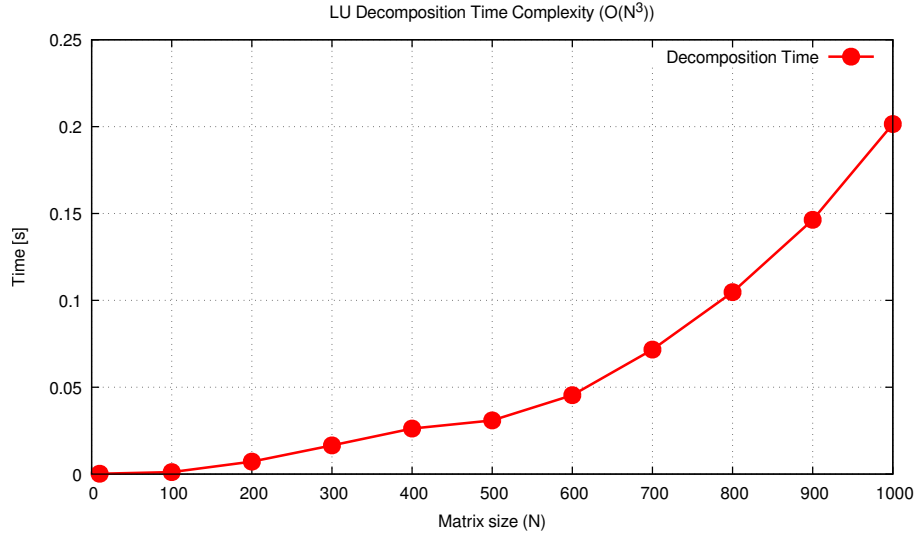


Figure 1: LU Decomposition time as a function of matrix size N . The curve follows an $O(N^3)$ trend.

In contrast, the solving phase (Figure 2) demonstrates a quadratic growth trend, remaining significantly faster than the decomposition phase even as N increases.

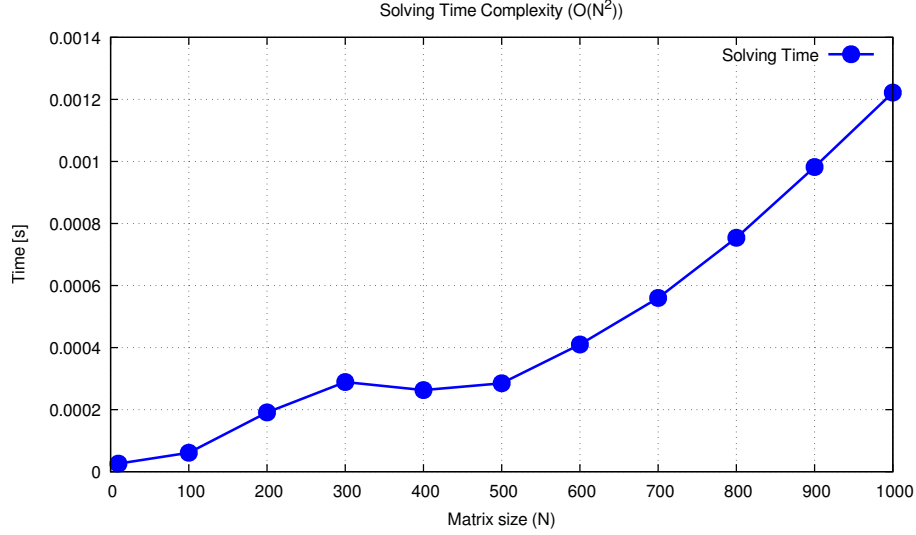


Figure 2: System solving time as a function of matrix size N . The trend is consistent with $O(N^2)$ complexity.

To rigorously quantify the scaling behavior, we compared the increase in execution time when the problem size N is doubled from 500 to 1000:

1. Decomposition Phase:

$$\frac{T_{decomp}(1000)}{T_{decomp}(500)} = \frac{0.201480}{0.030898} \approx 6.52$$

While the theoretical expectation for cubic complexity is $2^3 = 8$, the observed ratio is slightly lower. This deviation is likely attributable to hardware optimizations, such as CPU caching efficiency, which can have a more pronounced impact at smaller matrix sizes ($N = 500$) compared to larger ones. Nevertheless, the growth is decisively super-quadratic.

2. Solving Phase:

$$\frac{T_{solve}(1000)}{T_{solve}(500)} = \frac{0.001222}{0.000285} \approx 4.29$$

This result aligns closely with the theoretical prediction for quadratic complexity ($2^2 = 4$).

Figure 3 presents a combined logarithmic plot, highlighting the widening gap between the computational costs of the two operations.

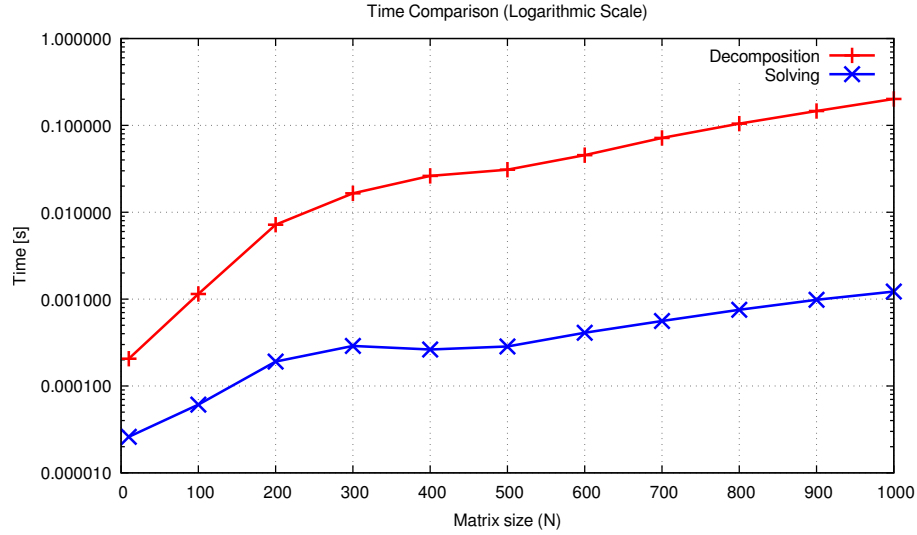


Figure 3: Logarithmic comparison of decomposition and solving times. The slope difference corresponds to the difference in complexity class.

4 Conclusion

The experimental results confirm the theoretical complexity estimates for linear system solving algorithms discussed in standard literature[1]. The LU decomposition step dominates the total computational cost, scaling as $O(N^3)$, while the substitution step scales as $O(N^2)$.

At $N = 1000$, the decomposition takes approximately 165 times longer than the solving step. This validates the standard numerical strategy: if one needs to solve multiple linear systems with the same coefficient matrix A but different vectors b , it is computationally efficient to perform the LU decomposition once and reuse the factorization for rapid $O(N^2)$ solutions.

Bibliography

- [1] Numerical algorithms, <http://deadline.3x.ro/numerical.html>, educational Materials
- [2] Galassi, M., Davies, J., Theiler, J., Gough, B., Jungman, G., Alken, P., Booth, M., Rossi, F.: GNU Scientific Library Reference Manual, 3rd edn. (2009)